LAB SESSION 10

Controls in C#

Objective:
To learn about use of different controls like buttons, text boxes, combo box and timer controls in C# applications and how to apply graphics and Mouse events.

Introduction:

Example:
Create a simple C# that will place one combo box filled with the names of different shapes by using item property of combo box and application should be able to draw the selected shape of combo box control by creating graphic ,pen and solid brush objects.

```
namespace ListboxnCombo
{
   public partial class Form1 : Form
   {
      public Form1()
      {
         InitializeComponent();
      }

      private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)
      {
         Graphics g = base.CreateGraphics();
         Pen p = new Pen(Color.Blue);
         SolidBrush sb = new SolidBrush(Color.Brown);
         g.Clear(Color.White);

         switch (comboBox1.SelectedIndex)
         {
            case 0:
               g.FillEllipse(sb, 50, 50, 150, 150);
               break;
            case 1:
               g.DrawRectangle(p, 50, 50, 150, 150);
               break;
            case 2:
               g.DrawEllipse(p, 50, 80, 150, 170);
               break;
            case 3:
```

```
                g.DrawRectangle(p, 50, 50, 50, 50);
                break;
            }

        }

    }
}
```

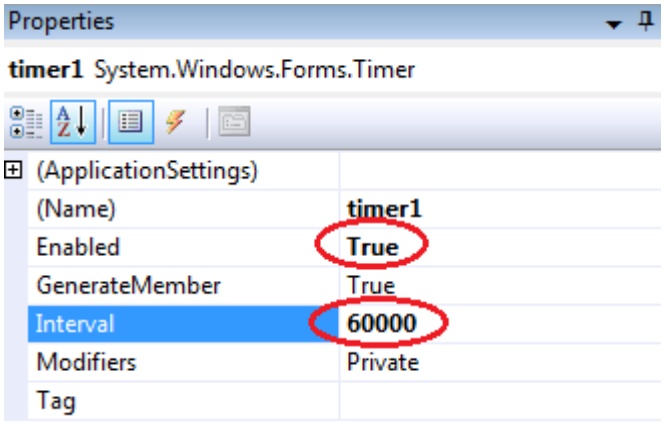
Timer Control:
he *Timer Control* plays an important role in the development of programs both Client side and Server side development. A **Timer** control raises an event at a given interval of time without using a secondary thread. If you need to execute some code after certain interval of time continuously, you can use a timer control.

Timer Class Properties

Enabled property of timer represents if the timer is running. We can set this property to true to start a timer and false to stop a timer.

Interval property represents time on in milliseconds, before the Tick event is raised relative to the last occurrence of the Tick event. One second equals to 1000 milliseconds. So if you want a timer event to be fired every 5 seconds, you need to set Interval property to 5000.

Timer t = new Timer();
Namespace: System.Timers
Assembly: System (in System.dll)



Task 2:
Create an array of Buttons that would display buttons dynamically at run time.
Sample Code:

```
private void Form1_Load(object sender, EventArgs e)
    {
      Button[] b = new Button[5];

      for (int i = 1; i <= b.Length-1; i++)
      {
        b[i] = new Button();
        b[i].Text = "Button" + i;
        b[i].Size = new Size(70,  30);
        b[i].Location = new Point(i + 70, i + 100);
        b[i].Left = 50*i;
        b[i].Top = i * 70;
        b[i].Click += new System.EventHandler(button1_Click);
        this.Controls.Add(b[i]);


      }
    }
```

Task 3:
Create a slideshow of pictures by using picture box control and timer control. Make a folder on any drive containing all of your pictures, you want to include in a slideshow. Rename all the pictures starting with 1.jpg, 2.jpg… n.jpg.

Task 4:
Using Mouse up, down and Mouse move events, Create an application that can draw freehand drawing.

Sample Code:

```
        int readypaint = 1;

    private void Form1_Load(object sender, EventArgs e)
    {
       timer1.Start();
    }

    private void Form1_MouseUp(object sender, MouseEventArgs e)
    {
       readypaint = 0;
    }

    private void Form1_MouseMove(object sender, MouseEventArgs e)
    {
       if (readypaint == 1)
       {
          Graphics g = CreateGraphics();
          Pen p = new Pen(Color.Red);
          g.DrawEllipse(p, e.X, e.Y, 10,10);
       }
    }

    private void Form1_MouseDown(object sender, MouseEventArgs e)
    {
       readypaint = 1;
    }
```

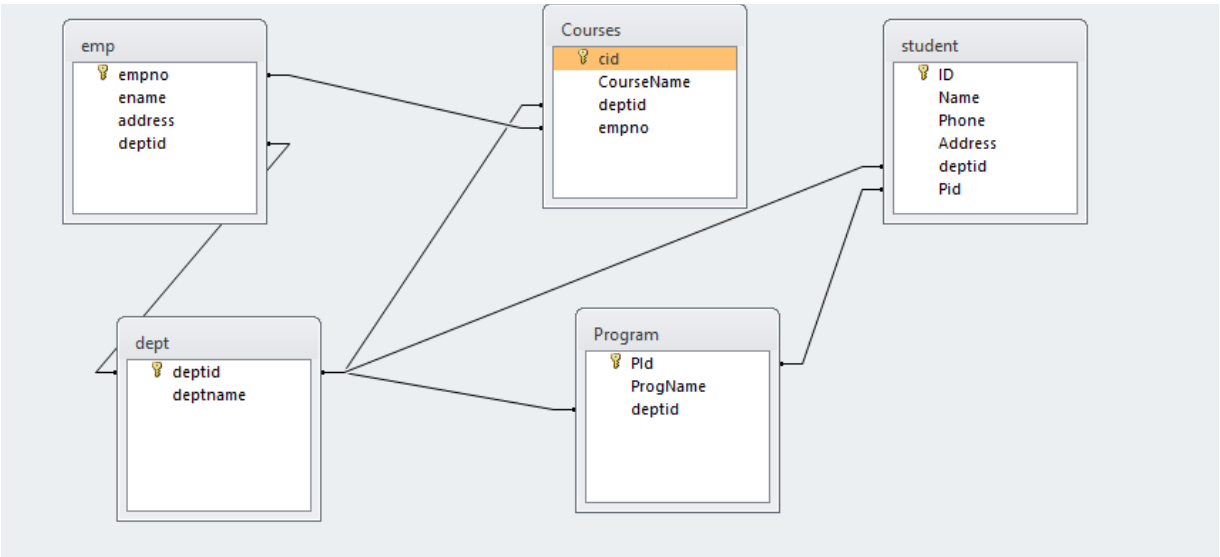
The main objective of this lab is to be able to use databases C# applications using **ADO.Net** classes and **Microsoft SQL Server and Microsoft Access**. We will use data access components tools to connect to, retrieve and update data in database. We will explore how we can leverage the built-in capabilities of ADO .NET to extract and manipulate data as well as insert, update and delete data in SQL Server. With this, we will take a look at **CurrencyManager** object to navigate the records in our bound controls.

Program 1
This simple application how to create database connectivity in C # application.

Steps

Create a Database consisting of four tables containing necessary attributes of these tables along with the connectivity's .

1. Department.
2. Teachers.
3. Student.
4. Subjects
5. Courses
6. Program



1. Start a new Windows Application project. Design GUI as shown below:

2. As this in untyped connection, we have to import System.Data.Oledb name space
3. We'll declare connection, Data Adapter, Data Set and an integer variable as globel:

```
OleDbConnection con = new OleDbConnection("Provider=Microsoft.jet.Oledb.4.0;Data Source= e:\SalesDatabase.mdb;");
 OleDbDataAdapter adap = new OleDbDataAdapter("select * from student", "Provider=Microsoft.jet.Oledb.4.0;Data Source=e:/std.mdb;");
  DataSet d = new DataSet("student");

  Int intCurrentIndex =0;
```

1. Data Adapter acts as a bridge between application and connection. For that a Data Adapter must know which database and what table(s) to use as we'll use Data Adapter to fill our Data Set: On form's load event, write following line of code:
2.

```
da.Fill(ds);
'DS is the name of Data Set
```

Do It Yourself: Code click events of **"<"** button and **">>"** button yourself.

Program 2
This simple application demonstrates How to Use DataGridView control to display data:

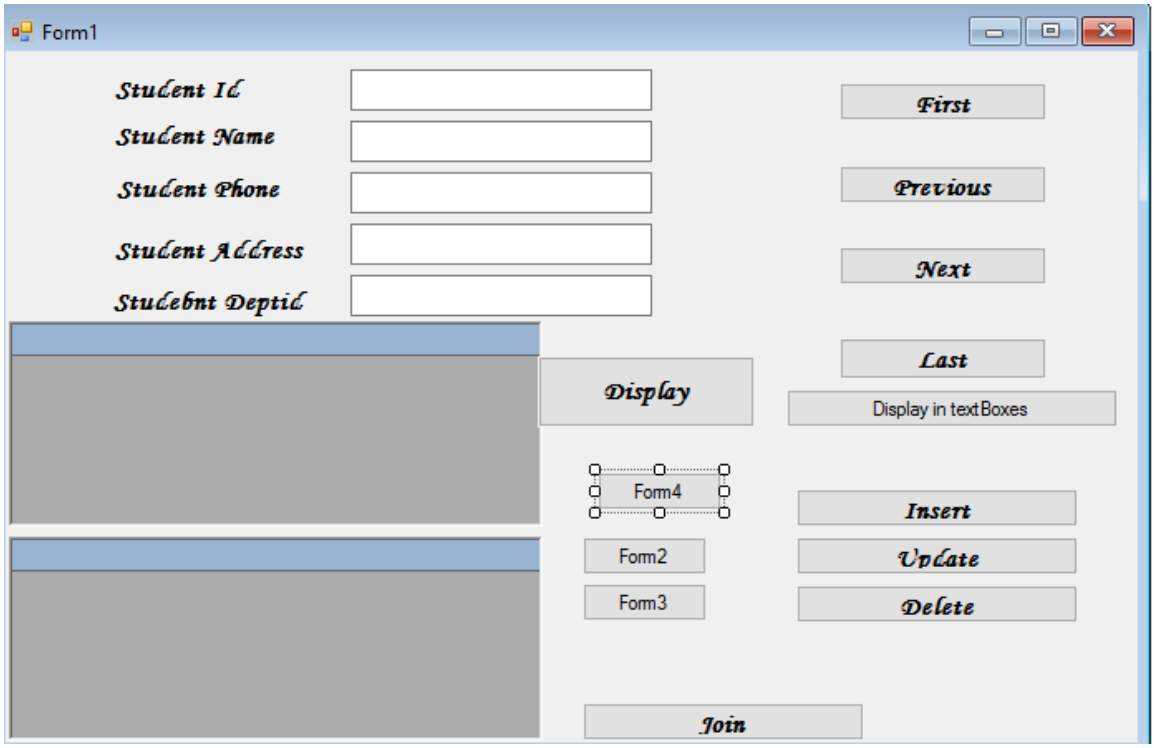
1. Open a new form and place DataGrid Control and a Button on your form.
2. As in previous example import Oledb namespace and declare Connection, Data Adapter and Data Set as global variables.
3. Go to Load Event of your for and enter the following coding:

```
string constr = "Provider=Microsoft.Jet.Oledb.4.0; Data source =e:/std.mdb";
OleDbConnection con = new OleDbConnection(constr);
        string strsql = "select * from student";
OleDbDataAdapter adap = new OleDbDataAdapter(strsql, con);
DataSet d1 = new DataSet("student");
con.Open();
adap.Fill(d1, "student");
dataGrid1.DataSource = d1;
```

4. Save and execute your work.

Program 3:

Enhance your form according to snapshot:(Data Manipulation)



Solution:

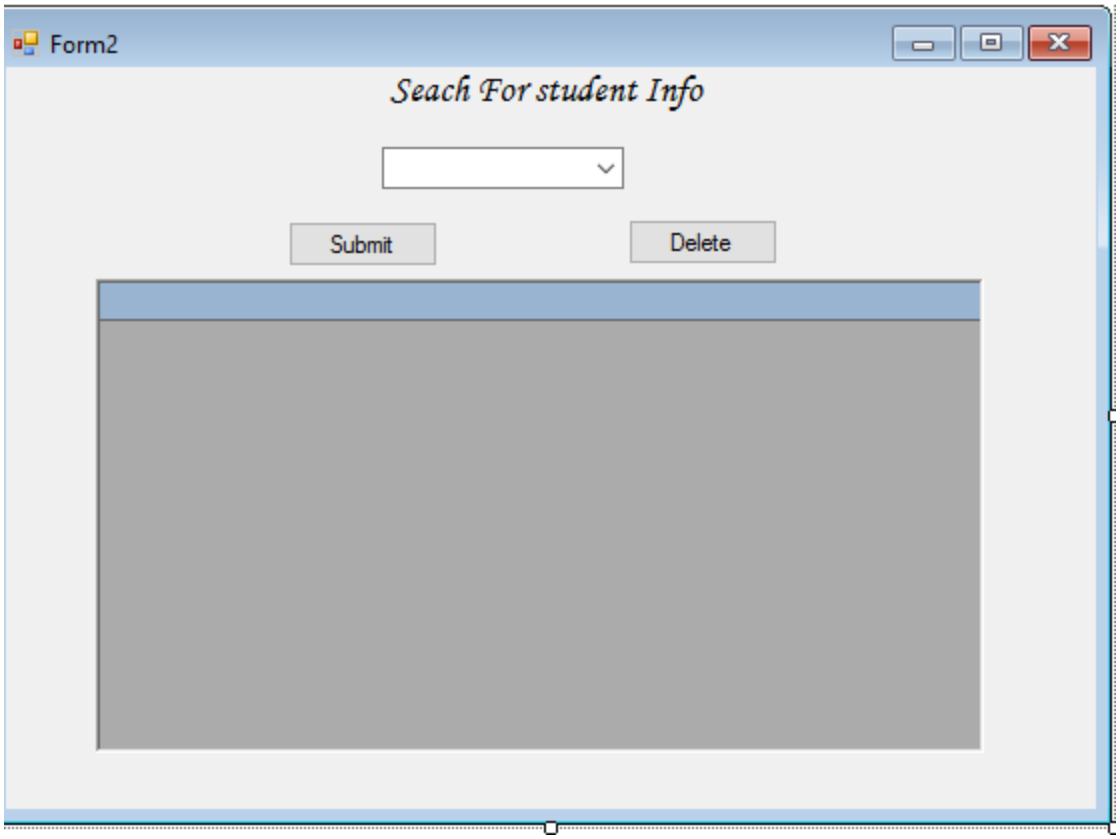
```
OleDbConnection con = new OleDbConnection("Provider=Microsoft.Jet.OLEDB.4.0;
Data source=f:/std.mdb");
    DataSet d1 = new DataSet("student");
    int counter = 0;
    private void button1_Click(object sender, EventArgs e)
    {


    }

    private void button6_Click(object sender, EventArgs e)
    {

      textBox1.Text = d1.Tables[0].Rows[0]["ID"].ToString();
      textBox2.Text = d1.Tables[0].Rows[0]["Name"].ToString();
      textBox3.Text = d1.Tables[0].Rows[0]["Phone"].ToString();
      textBox4.Text = d1.Tables[0].Rows[0]["Address"].ToString();
      textBox5.Text = d1.Tables[0].Rows[0]["deptid"].ToString();


    }
```

```
private void Form1_Load(object sender, EventArgs e)
{
   con.Open();

}

private void button2_Click(object sender, EventArgs e)
{
   if (counter > 0)
   {
      counter = 0;

      textBox1.Text = d1.Tables[0].Rows[counter]["ID"].ToString();
      textBox2.Text = d1.Tables[0].Rows[counter]["Name"].ToString();
      textBox3.Text = d1.Tables[0].Rows[counter]["Phone"].ToString();
      textBox4.Text = d1.Tables[0].Rows[counter]["Address"].ToString();
      textBox5.Text = d1.Tables[0].Rows[counter]["deptid"].ToString();

   }
}

private void button3_Click(object sender, EventArgs e)
{
   if (counter > 0)
   {
      counter = counter - 1;
      textBox1.Text = d1.Tables[0].Rows[counter]["ID"].ToString();
      textBox2.Text = d1.Tables[0].Rows[counter]["Name"].ToString();
      textBox3.Text = d1.Tables[0].Rows[counter]["Phone"].ToString();
      textBox4.Text = d1.Tables[0].Rows[counter]["Address"].ToString();
      textBox5.Text = d1.Tables[0].Rows[counter]["deptid"].ToString();
   }
   else
   {
      MessageBox.Show(" U are lready on the first record");
   }
}

private void button4_Click(object sender, EventArgs e)
{
   if (counter < d1.Tables[0].Rows.Count - 1)
   {
      counter = counter + 1;
      textBox1.Text = d1.Tables[0].Rows[counter]["ID"].ToString();
      textBox2.Text = d1.Tables[0].Rows[counter]["Name"].ToString();
```

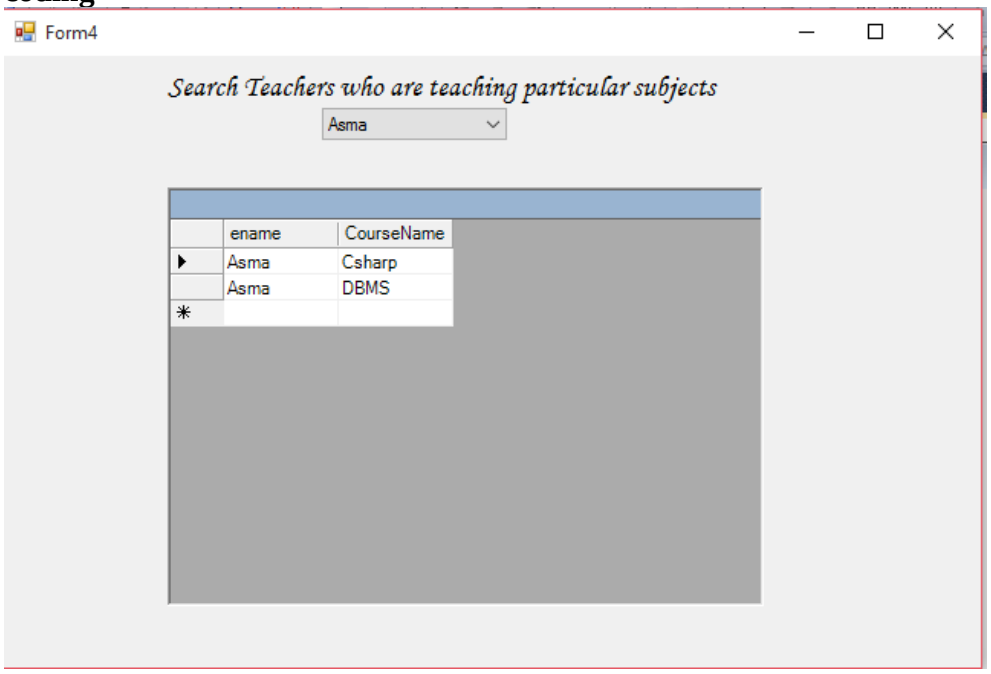
```
            textBox3.Text = d1.Tables[0].Rows[counter]["Phone"].ToString();
            textBox4.Text = d1.Tables[0].Rows[counter]["Address"].ToString();
            textBox5.Text = d1.Tables[0].Rows[counter]["deptid"].ToString();
        }
    }

    private void button5_Click(object sender, EventArgs e)
    {
        if (counter < d1.Tables[0].Rows.Count - 1)
        {
            counter = d1.Tables[0].Rows.Count - 1;
            textBox1.Text = d1.Tables[0].Rows[counter]["ID"].ToString();
            textBox2.Text = d1.Tables[0].Rows[counter]["Name"].ToString();
            textBox3.Text = d1.Tables[0].Rows[counter]["Phone"].ToString();
            textBox4.Text = d1.Tables[0].Rows[counter]["Address"].ToString();
            textBox5.Text = d1.Tables[0].Rows[counter]["deptid"].ToString();
        }
    }

    private void button7_Click(object sender, EventArgs e)
    {

        OleDbDataAdapter adap1 = new OleDbDataAdapter(" Select student.ID , dept.deptid from student inner join dept on dept.deptid=student.deptid ", con);
        DataSet d2 = new DataSet();
        adap1.Fill(d2,"student");
        dataGrid1.DataSource = d2;
        OleDbDataAdapter adap2 = new OleDbDataAdapter("select Courses.CourseName,emp.ename from emp inner join courses on emp.empno=courses.empno", con);

       adap2.Fill(d2,"emp");
        dataGrid2.DataSource=d2;


    }

    private void button8_Click(object sender, EventArgs e)
    {
        OleDbCommand com1 = new OleDbCommand("Insert into student(ID,Name,Phone,Address,deptid) values(' " + textBox1.Text + " ',' " + textBox2.Text + " ',' " + textBox3.Text + " ',' " + textBox4.Text + " ',' " + textBox5.Text + " ')", con);
        com1.ExecuteNonQuery();
        MessageBox.Show("One record has been added");
```

```
        }

        private void button10_Click(object sender, EventArgs e)
        {
            OleDbCommand com2 = new OleDbCommand("DELETE FROM student where
ID = '" + textBox1.Text + " '", con);
            com2.ExecuteNonQuery();
            MessageBox.Show("One record has been deleted ");


        }

        private void button11_Click(object sender, EventArgs e)
        {

            Form2 f2 = new Form2();
            f2.Show();
        }

        private void button9_Click(object sender, EventArgs e)
        {
            OleDbCommand com3 = new OleDbCommand("UPDATE student set
Address='Lahore' where ID= '" + textBox1.Text + " ' ", con);

            com3.ExecuteNonQuery();
            MessageBox.Show(" One record has been updated");


        }

        private void button12_Click(object sender, EventArgs e)
        {
            Form3 f3 = new Form3();
            f3.Show();

        }

        private void button13_Click(object sender, EventArgs e)
        {
            Form4  f4 = new Form4();
            f4.Show();
        }
    }
}
```

Task 1: Design this interface for second Form.



Task 2: **Design this interface for third form And write down necessary coding**



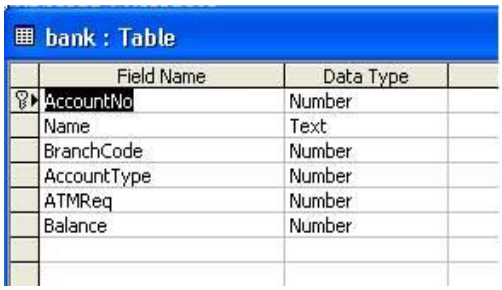
Program 5:

This database application demonstrates the use of ADO.Net classes using Access Database.Steps:

1. For this example we have created MS-ACCESS database named **Csharp1.mdb**. The table structure for the **bank** table used in this example is show below:

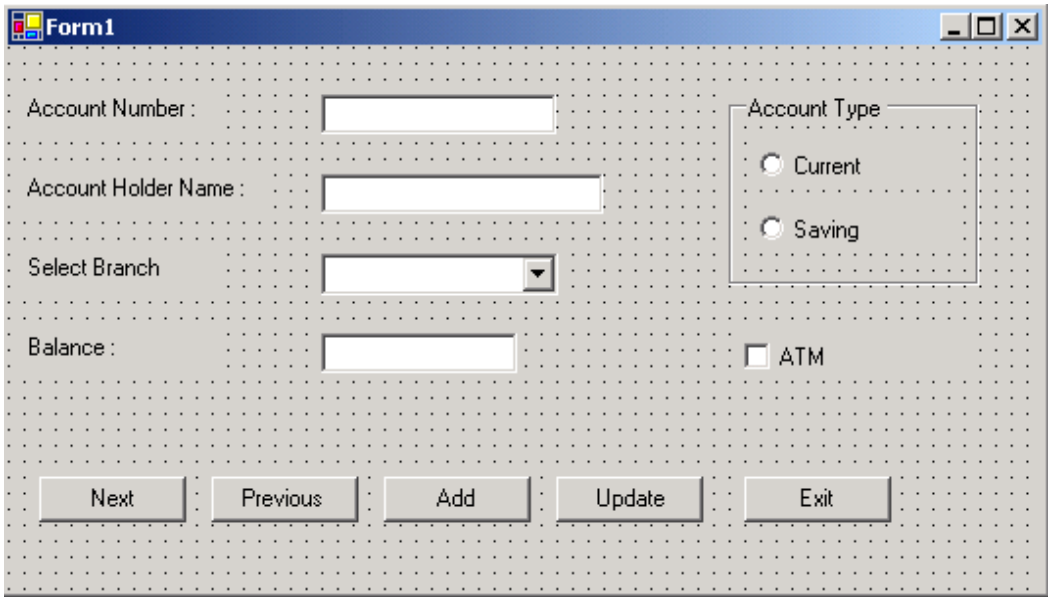
   The properties of the fields are as follows

   - AccountNo - Number - Long Integer
   - Name - string
   - BranchCode - int
   - AccountType - int
   - ATMReq - int
   - Balance - Number - Decimal



2. Create a new Windows Application project. From toolbox's Data tab, select OleDbDataAdapter component and configure it to establish connection with your database.

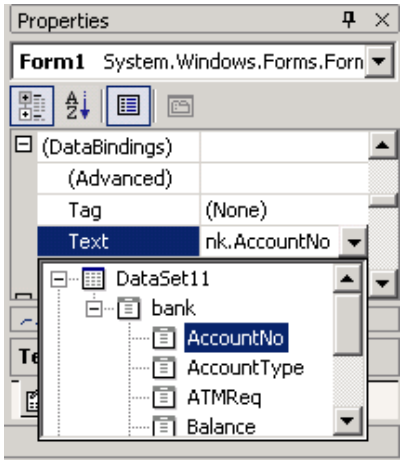
3. Design your GUI as shown below:



4. In this application, we will be working with individual data bound controls.

5. Select first Textbox control on your form.

6. Open **DataBinding** section on control's Properties window, locate the Text item, and expand its list of possible settings. Select the one you want to display in your control as shown below:



Week # 10

Composition & Aggregation in OOP

Objective:

Understand the concepts of composition and aggregation in Object-Oriented Programming (OOP) using C++ and learn how to implement these concepts in practical scenarios.

Introduction:

Composition and Aggregation are two types of associations that represent relationships between objects. Composition is a strong form of association with a strict lifecycle dependency between the parent and child objects. If the parent object is destroyed, the child objects are also destroyed. Aggregation is a weaker form of association where the child objects can exist independently of the parent object.

Feature	Composition	Aggregation
Definition	A strong form of association where the lifetime of the contained object is managed by the container object.	A weaker form of association where the contained objects can exist independently of the container object.
Lifetime Dependency	Contained objects do not exist independently. When the container is destroyed, contained objects are also destroyed.	Contained objects can exist independently. When the container is destroyed, contained objects may continue to exist.
Ownership	The container class owns the contained objects.	The container class does not own the contained objects.
Example	A `Person` class containing a `Heart` object. When the `Person` is destroyed, the `Heart` is also destroyed.	A `Library` class containing a collection of `Book` objects. The `Books` can exist independently of the `Library`.
Code Example (C++)	`cpp class Heart { /*...*/ }; class Person { private: Heart heart; public: Person() : heart() {} /*...*/ };`	`cpp class Book { /*...*/ }; class Library { private: vector<Book> books; public: void addBook(const Book& book) { books.push_back(book); } /*...*/ };`

UML Representation	Filled diamond (black diamond)	Empty diamond (white diamond)
Strength of Relationship	Strong relationship	Weak relationship
Reusability	Low reusability of the contained object outside the container	High reusability of the contained object outside the container
Implementation Complexity	Generally more complex due to lifecycle management	Generally simpler due to independent lifecycle management

Composition in C++

In composition, the contained objects are usually part of the parent object. This is often described as a "has-a" relationship.

Aggregation in C++

In aggregation, the contained objects can exist independently of the parent object. This is also a "has-a" relationship but with a weaker dependency.

Part 1: Composition

Create a Date class that contains day, month, and year.

Create a Person class that contains a name, age, and a Date object representing the birthdate.

```
#include <iostream>
using namespace std;

class Date {
private:
   int day, month, year;
public:
   Date(int d, int m, int y) : day(d), month(m), year(y) {}
   void display() {
      cout << day << "/" << month << "/" << year << endl;
   }
};
class Person {
private:
```

```
    string name;
    int age;
    Date birthdate;
public:
    Person(string n, int a, Date b) : name(n), age(a), birthdate(b) {}
    void display() {
        cout << "Name: " << name << ", Age: " << age << ", Birthdate: ";
        birthdate.display();
    }
};
int main() {
    Date birthDate(1, 1, 2000);
    Person person("John Doe", 24, birthDate);
    person.display();
    return 0;
}
```

Part 2: Aggregation

Create a Car class that contains a model, manufacturer, and year.

Create a Garage class that contains a collection of Car objects.

```
#include <vector>
#include <iostream>
using namespace std;

class Car {
private:
    string model;
    string manufacturer;
    int year;
public:
    Car(string m, string manu, int y) : model(m), manufacturer(manu), year(y) {}
    void display() {
```

```
        cout << "Model: " << model << ", Manufacturer: " << manufacturer << ", Year: " << year << endl;
    }
};



class Garage {
private:
    vector<Car> cars;
public:
    void addCar(Car car) {
        cars.push_back(car);
    }
    void displayCars() {
        for (auto &car : cars) {
            car.display();
        }
    }
};
int main() {
    Car car1("Model S", "Tesla", 2020);
    Car car2("Mustang", "Ford", 2018);

    Garage garage;
    garage.addCar(car1);
    garage.addCar(car2);

    garage.displayCars();
    return 0;
}
```

Choosing Between Composition and Aggregation

When designing a system, it is crucial to understand the relationship between objects to decide whether to use composition or aggregation:

Use composition when:

The contained object is part of the container object and its existence is meaningless outside the container.
You want to enforce a strong lifecycle dependency where the destruction of the container object should also destroy the contained objects.

Use aggregation when:

The contained objects can logically exist independently of the container.
You need to allow for the reuse of the contained objects in different contexts.

By carefully considering these factors, you can design your classes and their relationships in a way that best fits the logical and functional requirements of your system. This ensures that your code is both robust and maintainable, adhering to good object-oriented design principles.

Practice Questions

Question 1: Composition

Create a Book class that has title, author, and a Date object for the publication date. Implement a method to display the book details.

Question 2: Aggregation

Create a Library class that can hold multiple Book objects. Implement methods to add books and display all books in the library.

Question 3: Composition with push_back

Create a Playlist class that contains a collection of Song objects. Each Song object should have a title, artist, and duration. Implement methods to add songs and display the playlist.

Question 1: Composition
Create a Book class that has title, author, and a Date object for the publication date. Implement a method to display the book details.

```cpp
#include <iostream>
using namespace std;

class Date {
    private:
    int day, month, year;
    public:
        Date(int d, int m, int y) : day(d), month(m), year(y) {}
        void displayDate() {
            cout << day << "/" << month << "/" << year << endl;
        }

    };

class Book {
    private:
    string title, author;
    Date date;
    public:
    Book(string t, string a, Date d) : title(t), author(a), date(d) {}
    void displayBook() {
        cout << "Title: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "Publication date: ";
        date.displayDate();
    }
};

int main() {
    Date d1(10, 5, 2020);
    Book b1("The book", "The author", d1);
    b1.displayBook();
}
```

```
Title: The book
Author: The author
Publication date: 10/5/2020
```

Question 2: Aggregation

Create a Library class that can hold multiple Book objects. Implement methods to add books and display all books in the library.

```cpp
#include <iostream>
using namespace std;

class Book {
  private:
  string title, author;
  public:
    Book(string t, string a) : title(t), author(a) {}
    void displayBook() {
      cout << "Title: " << title << endl;
      cout << "Author: " << author << endl;
    }
    };

class Library {
  private:
  Book *books[100];
  int numBooks;
  public:
  Library() : numBooks(0) {}
  void addBook(Book *b) {
    books[numBooks] = b;
    numBooks++;
  }
  void displayAllBooks() {
    for (int i = 0; i < numBooks; i++) {
      books[i]->displayBook();
    }
  }
};

int main() {
  Library lib;
  Book b1("The book", "The author");
  Book b2("The book2", "The author2");
  lib.addBook(&b1);
  lib.addBook(&b2);
  lib.displayAllBooks();
}
```

```
Title: Lord of the rings
Author: Tolkien
Title: Toxic Positivity
Author: Whitney Goodman
```

Question 3: Composition with push_back

Create a Playlist class that contains a collection of Song objects. Each Song object should have a title, artist, and duration. Implement methods to add songs and display the playlist.

```cpp
#include <iostream>
#include <vector>
using namespace std;

class Song {
private:
    string title;
    string artist;
    int duration;
public:
    Song(string t, string a, int d) {
        title = t;
        artist = a;
        duration = d;
    }
    void displaySong() {
        cout << "Title: " << title << endl;
        cout << "Artist: " << artist << endl;
        cout << "Duration: " << duration << " seconds" << endl;
    }
};

class Playlist {
private:
    vector<Song> songs;
public:
    void addSong(Song s) {
        songs.push_back(s);
    }

    void displayPlaylist() {
        cout << "Songs in Playlist: \n\n";
        for (int i = 0; i < songs.size(); i++) {
            songs[i].displaySong();
            cout << endl;
        }
    }
};


int intro() {
    cout << "\nName : Syed Muhammad Rayyan   SE-23067 \n" << endl;
    return 0;
}

int songs() {
    Playlist p1;
    Song s1("wow wow", "Rayyan", 234);
    Song s2("Coaches dont play", "Danial", 320);
    Song s3("Enemeies", "Winston Churchill", 146);

    p1.addSong(s1);
    p1.addSong(s2);
    p1.addSong(s3);

    p1.displayPlaylist();

    return 0;
}

int main(int argc, char** argv) {
    intro();
    songs();
    return 0;
}
```

```
Name : Syed Muhammad Rayyan   SE-23067

Songs in Playlist:

Title: wow wow
Artist: Rayyan
Duration: 234 seconds

Title: Coaches dont play
Artist: Danial
Duration: 320 seconds

Title: Enemeies
Artist: Winston Churchill
Duration: 146 seconds
```

LAB SESSION 12

Friend Functions in C++

Objective

The objective of this lab is to understand and apply the concept of friend functions in C++. By the end of this lab, you should be able to define and use friend functions to access private and protected members of a class.

Introduction

In C++, friend functions are functions that are not member functions of a class but have access to the class's private and protected members. Friend functions are useful when you need to allow a non-member function to access private data of a class, which can be helpful in certain scenarios such as interfacing with external functions.

Theory

Friend Functions

A friend function is defined outside the class but has the right to access all private and protected members of the class. A friend function is declared using the keyword friend inside the class.

```
class ClassName {
   friend returnType friendFunctionName(arguments);
   // Other class members
};
```

Example:

```
#include <iostream>
using namespace std;

class Box {
private:
   double width;
public:
   Box(double w) : width(w) {}

   // Friend function declaration
   friend void printWidth(Box box);
};

// Friend function definition
void printWidth(Box box) {
   cout << "Width of box: " << box.width << endl;
}
```

```
int main() {
   Box box(10.5);
   printWidth(box);
   return 0;
}
```

In this example, printWidth is a friend function of the class Box and can access its private member width.

Conclusion

Friend functions provide a way to access the private and protected members of a class without being a member of the class. This can be particularly useful when you need to interface with external functions or perform operations that require access to private data.

Exercise:

1. Write a class Circle with a private member radius. Write a friend function to calculate the area of the circle.

2. Implement a class Rectangle with private members length and width. Write a friend function to calculate the perimeter of the rectangle.

1. Write a class Circle with a private member radius. Write a friend function to calculate the area of the circle.

```cpp
#include <iostream>
using namespace std;

class Circle {
    private:
    double radius;
    public:
        Circle(double r) : radius(r) {}
        friend void calculateArea(Circle c);
        };

void calculateArea(Circle c) {

    double area = 3.14 * c.radius * c.radius;
    cout << "Area of circle is: " << area << endl;
}

int main() {
    Circle c2(8.5);
    calculateArea(c2);
}
```

```
Area of circle is: 226.865

M:\C++\oel slabs 2\x64\Debug\oel slabs 2.exe
To automatically close the console when debug
```

2. Implement a class Rectangle with private members length and width. Write a friend function to calculate the perimeter of the rectangle.

```cpp
#include <iostream>
using namespace std;

class Box {
  private:
  double width;
  public:
    Box(double w) : width(w) {}
    friend void printWidth(Box b);
    };

void printWidth(Box b) {
  cout << "Width of box is: " << b.width << endl;
}

int main() {
  Box b2(9.7);
  printWidth(b2);
}
```

```
Width of box is: 9.7

M:\C++\oel slabs 2\x64\Debug\oel slabs 2.exe (p
To automatically close the console when debuggi
```

LAB SESSION 13

Operator Overloading in C++

Objective
The objective of this lab is to understand and apply the concept of operator overloading in C++. By the end of this lab, you should be able to overload operators to perform custom operations on objects of user-defined classes.

Introduction
Operator overloading allows the redefinition of the behavior of operators for user-defined types. This makes it possible to use operators like +, -, *, and others with class objects, making the code more intuitive and easier to understand.

Theory

Operator Overloading
Operator overloading allows you to define how operators work with class objects. The operator keyword is used to define an operator function.

Syntax:

```
returnType operator operatorSymbol(arguments);
```

Example

```
#include <iostream>
using namespace std;

class Complex {
private:
   float real;
   float imag;
public:
   Complex() : real(0), imag(0) {}
   Complex(float r, float i) : real(r), imag(i) {}

   // Overloading the '+' operator
   Complex operator + (const Complex& obj) {
      Complex temp;
      temp.real = real + obj.real;
      temp.imag = imag + obj.imag;
      return temp;
   }
```

```
    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(3.5, 2.5), c2(1.5, 4.5);
    Complex c3 = c1 + c2;
    c3.display();
    return 0;
}
```

In this example, the + operator is overloaded to add two Complex objects.

Conclusion

Operator overloading provides a powerful way to define custom operations for user-defined types. By overloading operators, you can make your code more intuitive and easier to read, while still performing complex operations on class objects.

Exercise:

1. Overload the * operator to multiply two Matrix objects (define a simple Matrix class with a 2D array).

Exercise:

1. Overload the * operator to multiply two Matrix objects (define a simple Matrix class with a 2D array).

```
#include <iostream>
using namespace std;

class Matrix {
  private:
  int arr[2][2];
  public:
    Matrix(int a, int b, int c, int d) {
      arr[0][0] = a;
      arr[0][1] = b;
      arr[1][0] = c;
      arr[1][1] = d;
    }
    void displayMatrix() {
      for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
          cout << arr[i][j] << " ";
        }
        cout << endl;
      }
    }
    Matrix operator*(Matrix m) {
      Matrix temp(0, 0, 0, 0);
      for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
          temp.arr[i][j] = arr[i][j] * m.arr[i][j];
        }
      }
      return temp;
    }
    };

    int main() {
      Matrix m1(1, 2, 3, 4);
      Matrix m2(9, 3, 1, 5);
      Matrix m3 = m1 * m2;
      m3.displayMatrix();
    }
```

```
Syed Muhammad Rayyan SE-23067
Matrix 3:
9 6
3 20
```